Inventory Management System for B2B SaaS

Overview

You're joining a team building "StockFlow" - a B2B inventory management platform. Small businesses use it to track products across multiple warehouses and manage supplier relationships.

Time Allocation

- **Take-Home Portion**: 90 minutes maximum
- **Live Discussion**: 30-45 minutes (scheduled separately)

Part 1: Code Review & Debugging (30 minutes)

A previous intern wrote this API endpoint for adding new products. **Something is wrong** - the code compiles but doesn't work as expected in production.

```
@app.route('/api/products', methods=['POST'])
def create_product():
    data = request.json

    # Create new product
    product = Product(
        name=data['name'],
        sku=data['sku'],
        price=data['price'],
        warehouse_id=data['warehouse_id']
    )

    db.session.add(product)
    db.session.commit()

    # Update inventory count
    inventory = Inventory(
        product_id=product.id,
        warehouse_id=data['warehouse_id'],
        quantity=data['initial_quantity']
    )

    db.session.add(inventory)
    db.session.commit()

    return {"message": "Product created", "product_id": product.id}
```

BASIC ERRORS:

Missing imports

```
  File "F:\Prabhleen New\practice\Backend Intern\backend.py", line 1, in <module>
    @app.route('/api/products', methods=['POST'])
NameError: name 'app' is not defined
```

Product,db,inventory not defined

```
    rv = self.dispatch_request()
  File "F:\Prabhleen New\practice\Backend Intern\venv\lib\site-packages\flask\app.py", line 902, in dispatch_request
    return self.ensure_sync(self.view_functions[rule.endpoint])(**view_args)  # type: ignore[no-any-return]
  File "F:\Prabhleen New\practice\Backend Intern\backend.py", line 17, in create_product
    product = Product(
NameError: name 'Product' is not defined
```

```
  File "F:\Prabhleen New\practice\Backend Intern\backend.py", line 37, in create_product
    inventory = Inventory(
NameError: name 'Inventory' is not defined
```

Therefore we add basic imports and definitions of classes:

```
from flask import Flask, request, jsonify
from flask_sqlalchemy import SQLAlchemy
from sqlalchemy.exc import IntegrityError

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///test.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
db = SQLAlchemy(app)

class Product(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String, nullable=False)
    sku = db.Column(db.String, unique=True, nullable=False)
    price = db.Column(db.Float, nullable=False)
     warehouse_id = db.Column(db.Integer)

class Inventory(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    product_id = db.Column(db.Integer, nullable=False)
    warehouse_id = db.Column(db.Integer, nullable=False)
    quantity = db.Column(db.Integer, nullable=False)
```

BUSINESS ISSUES:

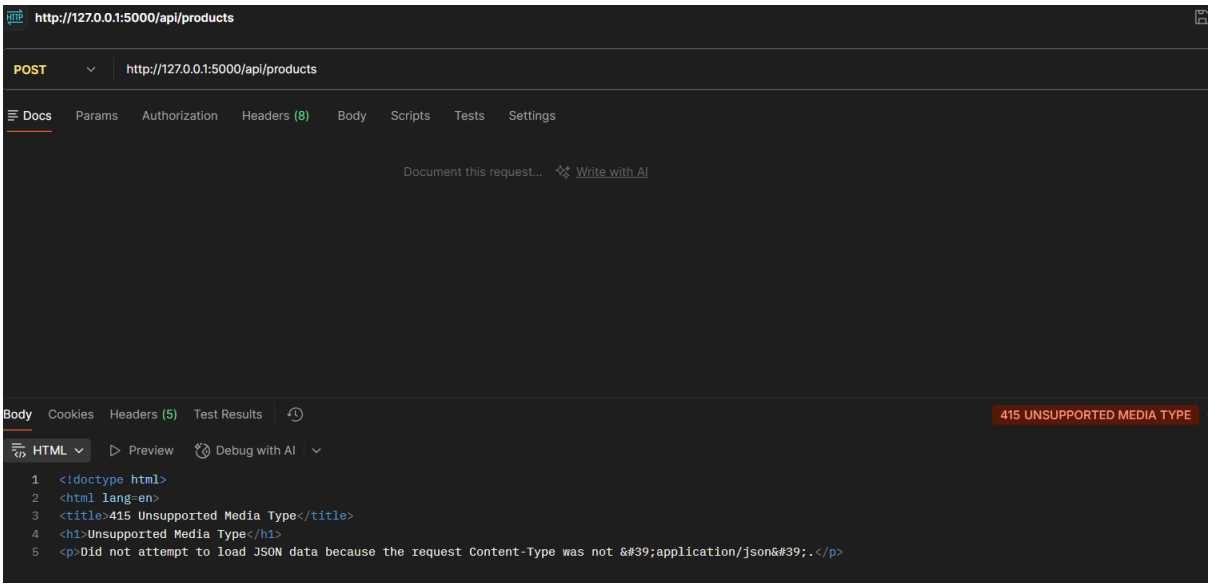
- Wrong stock numbers → selling items that are actually out of stock
- Same SKU for different products → orders getting mixed up
- Product only linked to one warehouse → can't use stock from other locations

- Negative quantity — system shows items available when they aren't
- Wrong prices — customers charged incorrect amounts
- Orders failing randomly — bad customer experience
- System crashes on simple mistakes — unreliable platform
- Hard to track inventory — confusion for warehouse teams
- Extra manual work — team has to fix data mistakes again and again
- Business can't grow easily — system doesn't support expansion

TECHNICAL ISSUES:

Issue-1: Missing Content-Type Validation

As we see if body is empty or not in json format,we'll get this error.



IMPACT:

1. Inconsistent API Behavior
2. Poor Developer Experience
3. Increased 500 Errors
4. Security & Stability Risk
5. Violates API Contract

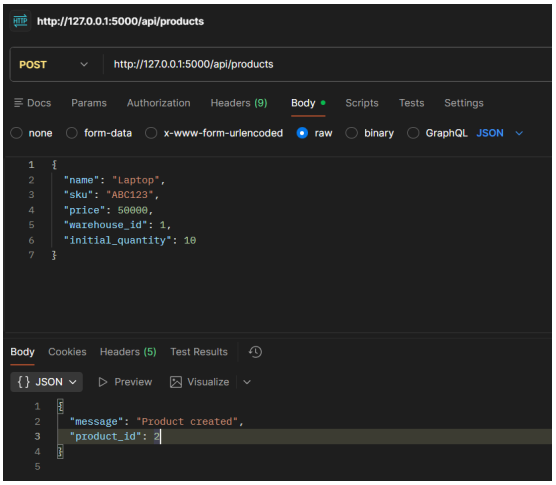
SOLUTION:

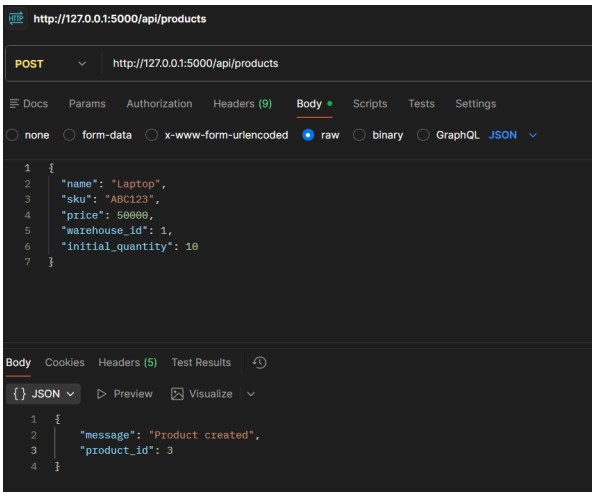
```
if not request.is_json:
    return jsonify({"error": "Send JSON data"}), 415
data = request.get_json()
if not data:
    return jsonify({"error": "Empty request body"}), 400
```

Validating Content-Type ensures strict adherence to API contracts and prevents malformed requests from reaching business logic.

ISSUE-2: SKU Uniqueness Not Enforced

Not checking for uniqueness of SKU when adding to the database





Same SKU but different product id's means duplicate sku is being saved in the database.

IMPACT:

- Data inconsistency → wrong product mapping in orders/inventory
- Billing & fulfillment errors due to ambiguous SKU reference
- Integration failures with external systems (ERP, logistics, analytics)

SOLUTION:

Make sku unique in the schema and run try-except blocks before committing in database.

ISSUE-3

Two separate commits for product and inventory.

So there is a possibility that:

1. Product saved
2. Inventory insert fails

Result:

- Product exists
- No inventory

- Database is inconsistent

Production Impact:

- Partial data writes → inconsistent state across tables
- Downstream failures (inventory, order processing, reporting)
- Difficult recovery/debugging due to lack of transactional integrity

SOLUTION:

Use flush()

- Sends data to DB temporarily
- Generates product.id
- BUT does **not commit**

<u>ISSUE-4:One Product to Multiple Warehouse Mapping</u>

Products can exist in multiple warehouses. But when we insert a new product into the database we use

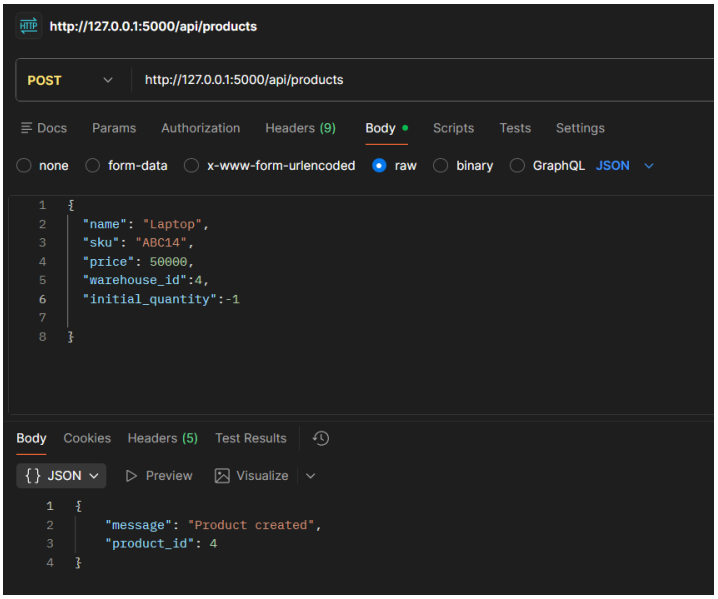
warehouse_id=data['warehouse_id']
Which means one product one warehouse.
Using an array to store warehouse Id's is not good as it's not good to store multiple values in a column.

Impact:

- Cannot scale to multi-warehouse inventory (blocks business expansion)
- Data redundancy / duplication of products across warehouses
- Incorrect stock tracking and reporting across locations

SOLUTION:
Removed warehouse_id from Product and used Inventory as a mapping table to support multiple warehouses per product, ensuring scalability and proper normalization.

ISSUE-5: No Validation for initial_quantity



The above screenshot shows that we are able to store negative quantity values in the database which is not correct.

Impact:

- Incorrect inventory tracking (negative stock levels)
- Order fulfillment issues (selling unavailable items)
- Misleading analytics and reporting

SOLUTION:

Validate before insertion in database.

```
try:
    quantity = int(data.get('initial_quantity', 0))
    if quantity < 0:
        return jsonify({"error": "Quantity cannot be negative"}), 400
except:
    return jsonify({"error": "Invalid quantity"}), 400
```

ISSUE-6: No Response Standardization

return {"message": "Product created", "product_id": product.id}

Impact:

- No HTTP status codes
- Hard to debug client-side

SOLUTION:

Return a proper JSON response with HTTP status code using jsonify.

ISSUE-7: Improper Exception Handling

No try-except blocks for exception handling have been used.

Impact

- Wrong HTTP responses (400 vs 500 confusion)
- Hard to debug issues
- Poor API reliability

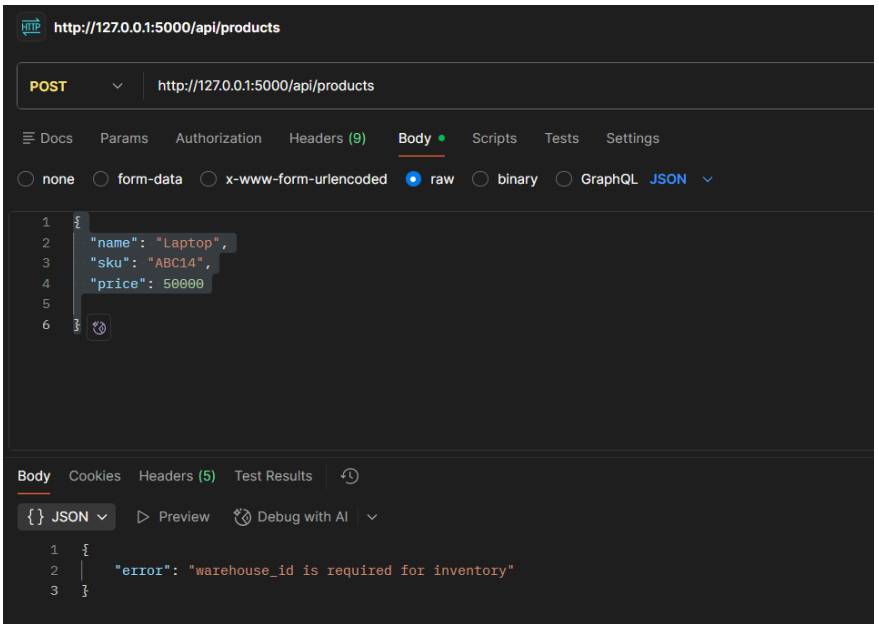
ISSUE-8: Improper Handling of Optional Fields

Requirement says: Some fields are optional

But earlier: warehouse_id = data['warehouse_id']

This Causes:

- KeyError if missing
- Or DB errors if passed as None



Impact

- API crashes on valid requests
- Violates flexibility requirement
- Poor user experience

Solution

We consider 'name', 'sku', 'price' as required field,rest as optional

Use .get() + conditional logic

```
warehouse_id = data.get('warehouse_id')
if warehouse_id is not None:
  # only then create inventory
```

FINAL CORRECTED CODE:

```
from flask import Flask, request, jsonify
from flask_sqlalchemy import SQLAlchemy
from sqlalchemy.exc import IntegrityError
app = Flask(__name__)


app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///test.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
db = SQLAlchemy(app)

# ISSUE-4 FIX: Removed warehouse_id from Product Schema
class Product(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String, nullable=False)
    sku = db.Column(db.String, unique=True, nullable=False)  # ISSUE-2 FIX
    price = db.Column(db.Float, nullable=False)

class Inventory(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    product_id = db.Column(db.Integer, nullable=False)
    warehouse_id = db.Column(db.Integer, nullable=False)
```

```
    quantity = db.Column(db.Integer, nullable=False)


@app.route('/api/products', methods=['POST'])
def create_product():


    # ISSUE-1: Content-Type validation
    if not request.is_json:
        return jsonify({"error": "Send JSON data"}), 415
    data = request.get_json()
    if not data:
        return jsonify({"error": "Empty request body"}), 400


    # ISSUE-8: Handle required fields properly
    required = ['name', 'sku', 'price']
    for field in required:
        if field not in data:
            return jsonify({"error": f"{field} is required"}), 400


    # ISSUE-5: Validate initial_quantity
    try:
        initial_quantity = int(data.get('initial_quantity', 0))
        if initial_quantity < 0:
            return jsonify({"error": "Quantity cannot be negative"}), 400
    except:
        return jsonify({"error": "Invalid quantity"}), 400
```

```
try:
    # Create product
    product = Product(
        name=data['name'],
        sku=data['sku'],
        price=data['price']
    )
    db.session.add(product)

    # ISSUE-3 FIX: use flush instead of commit
    db.session.flush()

    # ISSUE-8 FIX: Only create inventory if warehouse_id exists
    if data.get('warehouse_id') is not None:
        inventory = Inventory(
            product_id=product.id,
            warehouse_id=data['warehouse_id'],
            quantity=initial_quantity
        )
        db.session.add(inventory)

    # Single commit → atomic transaction
    db.session.commit()

# ISSUE-7 FIX: specific error handling
except IntegrityError:
```

```
        db.session.rollback()
        return jsonify({"error": "SKU already exists"}), 400

    except Exception:
        db.session.rollback()
        return jsonify({"error": "Internal server error"}), 500

    # ISSUE-6 FIX: Proper response + status code
    return jsonify({
        "message": "Product created",
        "product_id": product.id
    }), 201


if __name__ == '__main__':
    with app.app_context():
        db.create_all()
    app.run(debug=True)
```

Part 2: Database Design (25 minutes)

Based on the requirements below, design a database schema. **Note**: These requirements are intentionally incomplete - you should identify what's missing.

Given Requirements:

- Companies can have multiple warehouses
- Products can be stored in multiple warehouses with different quantities
- Track when inventory levels change
- Suppliers provide products to companies
- Some products might be "bundles" containing other products

DATABASE SCHEMA:

1. Companies

- companies table stores company details.
- Tracks creation timestamp.

2. Warehouses

- Linked to a company (company_id).
- Stores warehouse name, location, and creation timestamp.
- Allows **multiple warehouses per company**.

3. Products

- Stores product name, SKU, price, and whether it is a bundle (is_bundle).
- Tracks creation timestamp.

4. Inventory Management

- inventory table tracks **quantity of each product in each warehouse**.
- Unique constraint ensures **one record per product-warehouse pair**.
- updated_at timestamp for tracking last update.

5. Inventory Logs

- inventory_logs table tracks **changes in inventory**.
- Records change, new_quantity, reason, and timestamp.
- Provides **audit trail** for stock movements.

6. Suppliers

- Stores supplier details (name, contact_info).
- Linked to products via supplier_products.
- supplier_products table tracks **which supplier provides which product** and at what cost_price.

7. Product Bundles

- product_bundles table allows a **product to be composed of other products**.
- Self-referencing structure: bundle_id → product_id.
- Supports quantity of each component in the bundle.

8. Orders

- orders table tracks **customer orders**.
- Stores customer_name, status, total_amount, and creation timestamp.
- Linked to the company fulfilling the order.

9. Order Items

- order_items table stores **products in each order**.
- Tracks quantity and price at the time of order.
- Linked to orders and products.

<u>Key Reasons for things added into the schema:</u>

- **Inventory logs** – inventory_logs table to track changes over time with reason and timestamp.
- **SKU & price for products** – Helps with business/product management.
- **Cost price per supplier-product** – supplier_products.cost_price gives insight into margins.
- **Self-referencing bundle table** – product_bundles allows products to be bundles of other products.
- **Timestamps (created_at, updated_at)** – Added across multiple tables for tracking creation and updates.
- **Unique constraints** – Prevents duplicate entries in inventory, supplier_products, product_bundles.
- **Warehouse location info** – Helps in logistics planning.
- **Reason for inventory change** – Makes inventory_logs more informative for auditing.

IDENTIFYING GAPS:

Inventory & Warehouses

1. Should we **track minimum/maximum stock levels** per product or warehouse for automatic alerts?
2. Do we need to **support warehouse transfers** (moving inventory between warehouses)?
3. Are there **different types of inventory transactions** beyond sale/restock/adjustment (e.g., returns, spoilage, theft)?
4. Should we **track inventory reservations** for pending orders?
5. Do warehouses have **capacity limits** we need to enforce?

Products & Bundles

6. Should bundles be **sold only as bundles**, or can their components also be sold separately?
7. Do we need to **track historical prices** for products and bundles?
8. Are there **product categories, brands, or tags** for filtering and analytics?
9. Should bundles support **nested bundles** (bundles within bundles)?

Suppliers & Procurement

10. Do suppliers have **lead times, delivery schedules, or reliability metrics** we need to track?
11. Should we track **multiple suppliers per product with priority/preference**?
12. Are **supplier contracts or agreements** relevant (e.g., discount terms, minimum order quantities)?

Orders & Customers

13. Are orders **always linked to companies**, or do we also sell directly to end customers?
14. Should we track **shipping addresses, billing info, and delivery status**?
15. Do we need to support **partial shipments or backorders**?
16. Should we track **returns, cancellations, or refunds**?

Pricing & Financials

17. Are there **discounts, promotions, or dynamic pricing** rules to consider?
18. Should we track **cost vs selling price for profit analytics**?
19. Are **taxes, duties, or multi-currency support** needed?

Analytics & Reporting

20. Do we need **sales reports, inventory reports, or supplier performance dashboards**?
21. Should we track **KPIs like stock turnover, order fulfillment time, or margin per product**?

22. Are there **audit requirements** for all inventory or order changes?

Users & Permissions

23. Will multiple users/employees access this system? Should we track **roles and permissions**?
24. Do we need **audit trails of who made changes** (inventory, orders, products)

Lead Time Consideration:

25. Do we need to store 'Lead Time' (how long a supplier takes to deliver) in the supplier_products table?

Currency & Localization:

26. For a B2B SaaS, are we supporting multiple currencies for cost_price and total_amount?

POSSIBLE IMPROVEMENTS IN DATABASE DESIGN:

1. add a composite index on inventory(product_id, warehouse_id) to speed up the low-stock alert query.
2. we shouldn't physically delete products or warehouses. We should use a deleted_at timestamp so we don't break the history of inventory_logs.

Part 3: API Implementation (35 minutes)

Implement an endpoint that returns low-stock alerts for a company.

Business Rules (discovered through previous questions):

- Low stock threshold varies by product type
- Only alert for products with recent sales activity
- Must handle multiple warehouses per company
- Include supplier information for reordering

Endpoint Specification:

GET /api/companies/{company_id}/alerts/low-stock

Expected Response Format:

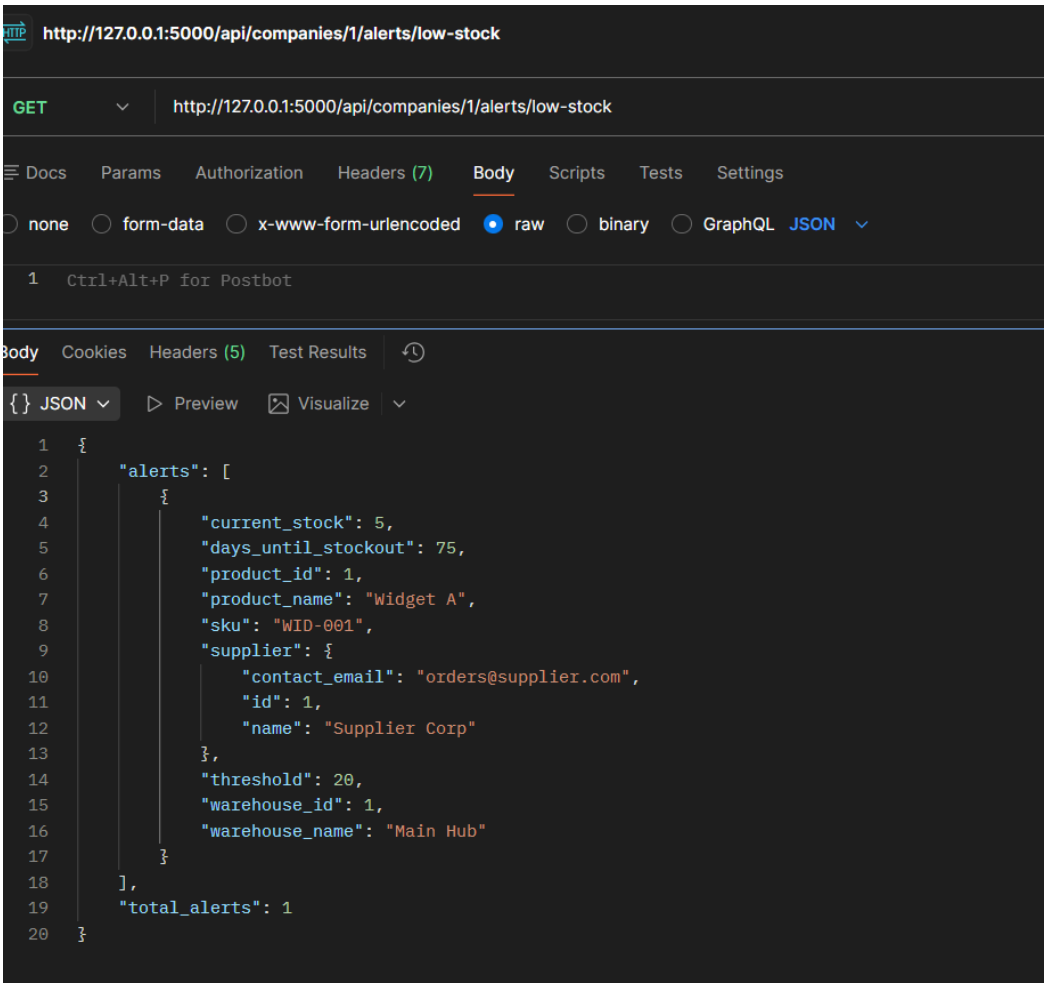
```
{
  "alerts": [
    {
      "product_id": 123,
      "product_name": "Widget A",
      "sku": "WID-001",
      "warehouse_id": 456,
      "warehouse_name": "Main Warehouse",
```

```
      "current_stock": 5,
      "threshold": 20,
      "days_until_stockout": 12,
      "supplier": {
        "id": 789,
        "name": "Supplier Corp",
        "contact_email": "orders@supplier.com"
      }
    }
  ],
  "total_alerts": 1
}
```

Here are the assumptions made based on business rules:

- **Low Stock Threshold:** Since the schema from previous question didn't have a threshold column, I've assumed a product_types table exists or that products has a min_stock_level and category field to determine the variable threshold.
- **Recent Sales Activity:** I've defined "recent" as any product appearing in the orders table within the last **30 days**.
- **Days Until Stockout:** This is calculated by taking the current_stock and dividing it by the average_daily_sales (total sales in the last 30 days / 30).
- **Primary Supplier:** While a product can have multiple suppliers, the alert will return the first one found in supplier_products.

OUTPUT:



IMPROVEMENTS:

For the GET /api/companies/{company_id}/alerts/low-stock endpoint, a large company might have 10,000 alerts. Suggest adding limit and offset parameters to the API.